/ /

Codigo ASCII:

El codigo ASCII Estandar Estadounidense para el intercambio de Informacion tiene sus raices a principio del siglo XX, en la era del telegrafo y codigos como el murray (de 5 bits). Sin embargo su version formal fue publicada por primera vez en 1963 por la asociacion Estadounidense de Estandars (ASA), precursora del actual ANSI. Un comite liderado por Robert W. Bemer fue clave en su desarrollo para estandarizar la representacion de datos y facilitar la comunicacion entre diferentes dispositivos y sistemas informaticos de la epoca. La version que se consolido como estandar universal se publico finalmente 1968.

Funcionamiento:

Funciona asignando un codigo binario unico de 7 bits a cada caracter de su conjunto. Con 7 bits, se puede generar 128 combinaciones distintas (2^7), que van del 0 al 127

Estructura:

El conjunto de 128 caracteres se dividen en dos grupos principales:

- Caracter de control (0-31 y 127): Son caracteres no imprimibles diseñados para controlar el flujo de datos o el formato del texto. Algunos 2 y son:

0(NUL): Usado como terminar de cadenas en lenguajes de programacion.

10(LF, Line Feed): Para un salto de linea

13(CR, Carriage Return): Para retorno de carro

27(ESC) Para iniciar secuencia de escape.

Caracteres imprimibles (32-126): Incluye:

- El espacio (32)
- Los digitos del 0 al 9 (48-57)
- Las letras mayuscula de la A (A) la Z (Z) (65-90)

Las letras minuscula de la a (a) a la z (z) (97-122)

- Signos de Puntuacion y simbolos matematicos (ej. !, #, $, *, +, -, ect.)

Utilidad:

Estandarizacion y Eficiencia: ASCII fue el primer estandar universal que permitio una representacion uniforme de datos en diferentes dispositivos. Su diseño de 7 bits era simple y eficiente para las limitadas capacidades de procesamiento y almacenamiento de las primeras computadoras.

Interoperabilidad: Su adopcion masiva lo convirtio en un lenguaje comun para computadoras, crucial para el desarrollo de las primeras redes en internet.

Legado y Compatibilidad: Es la base sobre la que se construyeron esquemas posteriores. El estandar moderno UTF-8 es totalmente compatible con ASCII, lo que significa que cualquier texto es tambien un texto UTF-8 valido. Esto garantiza la longevidad de los datos codificados en ASCII

Limitacion: Su principal limitacion es que solo cubre el alfabeto ingles, careciendo de soporte para caracteres acentuados, la letra 'ñ', o alfabetos

como cirílico, griego o los ideogramas asiáticos

UTF-8: El estándar Universal del Internet

A principios de la década de 1990, la necesidad de un estándar global que superara las limitaciones de ASCII era evidente. Tras intentos previos como UCS-2 y UTF-1 que fracasaron por no ser completamente compatible con ASCII, surgió la solución definitiva. El 2 de septiembre de 1992 Ken Thompson diseñó e implementó, junto Rob Pike, el formato UTF-8 mientras trabajaban en el sistema operativo Plan 9 de los Laboratorios Bell. Se basaron en un borrador anterior de Dave Prosser (FSS-UTF) y lograron crear una codificación que mantenía la compatibilidad con ASCII, era eficiente y autosincronizable. Fue presentado oficialmente en enero de 1993.

Funcionamiento:

UTF-8 es un formato de codificación de longitud variable que utiliza unidades de 8 bits (1 byte). Puede representar todos los más de un millón de puntos de código de Unicode utilizando secuencias de 1 a 4 bytes. Su estructura es la siguiente.

Rango de Puntos de Código Unicode	Nº de Bytes	UTF-8
000000 - 00007F	1 byte	0xxxxxx
000000 - 0007FF	2 bytes	110yyyyy 10xxxxxx
000800 - 00FFFF	3 bytes	1110zzzz 10yyyyyy 10xxxxxx
o 010000-10FFFF	4 bytes	11110uuu 10uuzzzz 10yyyyyy 10xxxxxx

Ej de codificación: La letra 'ñ' (Punto de código Unicode U+00F1) no está en ASCII. Su valor se encuentra en el rango de 2 bytes. Siguiendo el formato 110xxxxx 10xxxxxx, los 11 bits del punto de código se distribuyen, resultando en los bytes 11000011 10110001, que en hexadecimal son 0xC3 0xB1.

Autosincronización: Los prefijos de los bytes (0, 1110, 11110) permiten que un programa encuentre el inicio de un carácter aunque empiece a leer en medio de una secuencia de bytes.

Bom (Byte Mark): En UTF-8, la marca de orden de bytes (0xEF, 0xBB, 0xBF) no es necesaria para indicar el orden de bytes, ya que la unidad es el byte. Puede usarse

/ /

opcionalmente para indicar que un texto es UTF-8, pero no es recomendable por posibles problemas de compatibilidad.

Utilidad

Compatibilidad con ASCII: Es su mayor ventaja. Permite una transición suave desde sistemas heredados y hace que el software escrito para ASCII funcione sin modificaciones con UTF8.

Eficiencia de Espacio: Es extremadamente eficiente para textos en alfabeto latino (1 byte por caracter) y razonablemente eficiente para otros sistemas de escritura (2-3 bytes). Esto lo hace ideal para almacenamiento y transmision.

Sin problemas de Endianness: Al trabajar con bytes, no sufre de la ambiguedad entre "Big-endian" y "little-endian" que afecta a UTF-16

UTF-16 Optimizado para Representacion Interna

UTF-16 es la evolucion directa de UCS-2, un formato de codificacion de los primeros estandares Unicode (version 1.1) que usaba 2 bytes fijos para cada caracter. UCS-2 estaba limitado a representar solo Plano Basico Multilingue, que contenia los 65,536 caracteres mas comunes. Cuando Unicode se expandio en la version 2.0 para incluir mas planos (mas alla del U+FFFF) se hizo necesario un nuevo formato. Asi nacio UTF-16 que mantuvo la representacion de bytes 2 para BMP, pero añadio un mecanismo de pares sustitutos de 4 bytes para representar los nuevos caracteres de los planos suplementarios.

Funcionamiento:

- Para el plano basico Multilingue (BMP): Cualquier caracter con un punto de codigo en el rango U+0000 a U+FFFF se codifica directamente como un unico valor de 16 bits. Esto incluye la gran mayoria de los sistemas de escritura modernos, como alfabeto latino, cirilico, griego, arabe, hebreo, asi como los miles de ideogramas CJK (China, Japon y Corea).

/ /

• Para los planos suplementarios:
Los caracteres con punto de código superior se presentan mediante par sustituto de dos unidades de 16 bits (4 bytes en total). El primer elemento del par llamado "Subrogado alto" toma un valor en el rango reservado [D800, DBFF], y el segundo en el rango [DC00, DFFF]. Estos rangos están especialmente reservados y no se usan para caracteres individuales lo que garantiza que no haya ambigüedad.

• Endianness y Bom: Al trabajar con palabras de bytes, el orden de los (Bytes, 0 ds) es crucial. Un mismo valor de 16 de bits (ej. 0x425 puede almacenarse como 4E 25 (Big-Endian) o 25 4E (Little-Endian). Por ello, los archivos UTF-16 suelen comenzar con una marca de orden de Bytes (BOM) (U+FEFF) para indicar el endianness utilizado.

Utilidad:

Eficiencia para idiomas Asiáticos: es muy eficiente para representar texto en en idiomas como chino, japonés o coreano ya que su carácter se encuentra en la mayoría BMP y por

tanto ocupan solo 2 bytes. En UTF-8, en cambio mismo caracteres ocupan 3 bytes.

- Representación internos de sistemas y lenguajes: Por su eficiencia de lenguaje de menor utilización de caracteres en el BMP, fue adoptado como formato interno por plataformas clave como Java (en sus inicios), C# y el sistema operativo Windows (para la API nativa). En estos entornos, las cadenas de texto se manejan internamente en UTF-16.

Simplidad en el BMP: Para aplicaciones que solo trabajan con BMP, puede ser tratado como un formato de ancho fijo, lo que simplifica el procesamiento.

Aplicaciones del UTF-8

Gracias a sus propiedades, UTF-8 se ha vuelto se ha convertido en la codificación dominante en multiples ambitos:

Word Wide Web: En, sin diferencia la codificación mas utilizada. Segun un informe de W3Techs de abril de 2025, el 98,6 % de todos los sitios web usan UTF-8.

se especifica en cabeceras HTTP o metaetiqueta HTML

Correo Electrónico: El Internet Mail Consortium recomienda que todos los programas sean capaces de usar UTF-8 para la creación y visualización de mensajes, permitiendo el intercambio global de correos en cualquier idioma.

Protocolo de Internet: La IETF, requiere que todos los protocolos de internet indiquen la codificación que usan y que UTF-8 sea una de las opciones compatibles

Archivo de Configuración y Código Fuente: La mayoría de los editores de textos, entornos de desarrollo (IDEs) y sistemas operativos basados en Unix/Linux usan guardan los archivos de configuración y código fuente en UTF-8 por defecto

Bases de Datos: Sistema como MySQL, PostgreSQL o MongoDB permiten y recomiendan el uso de UTF-8 en MySQL extra soporte completo de 3 bytes para garantizar el almacenamiento de texto multilingüe

Por que el UTF-8 se ha convertido en el estandar dominante.

Se debe a una combinacion de factores tecnicos y practicos que lo hicieron ideal para el intercambio de informacion en gran escala, especialmente en internet

Recompatibilidad con ASCII: Es la razon fundamental. Permitio una adopcion masiva e inmediata sin romper los sistemas existentes. Cualquier texto ASCII es automaticamente valido en UTF-8.

Eficiencia en el contexto global: Esta optimizado para el texto en ingles y lenguas del alfabeto latino, que historicamente dominaban el codigo de programacion, las etiquetas HTML y gran parte del contenido de internet, usando solo un byte. Es eficiente en el caso de uso comun.

Ausencia de ambiguedad (Endianness): Al operar con bytes (endianness) de UTF-16 lo que significa el intercambio de datos en diferentes sistemas y elimina la necesidad de un BOM problematica

/ /

Detección de errores y autosincronización: Su estructura permite detectar fácil datos corruptos y en caso de pérdida de un byte, es posible re-sincronizarse con el inicio del siguiente carácter válido.